
SSJ approach and toolkit

Matthew Rognlie

Goethe Heterogeneous-Agent Macro Workshop, 2026

This lecture

- ❖ So far: built SIM model and HANK, applied to fiscal and monetary policy
- ❖ Until now, we've written our own code for every step:
 - ❖ Solved for steady state
 - ❖ Implemented fake news algorithm to obtain Jacobians
 - ❖ Hand-derived Jacobian system, solved to obtain impulse responses
- ❖ Today, introduce the **SSJ toolkit** to automate much of this process

What is the SSJ toolkit?

- ❖ Started in 2019 as code accompanying the Sequence-Space Jacobian paper
- ❖ Major revisions in 2020, 2021, 2022 to turn it into a bona fide “toolkit”
- ❖ Less active development in recent years, but planning for big push soon
 - ❖ Lots of new and improved methods to incorporate!

Sequence-Space Jacobian (SSJ)

Requirements and installation

SSJ runs on Python 3.7 or newer, and requires Python's core numerical libraries (NumPy, SciPy, Numba). We recommend that you first install the latest [Anaconda](#) distribution. This includes all of the packages and tools that you will need to run our code.

To install SSJ, open a terminal and type

```
pip install sequence-jacobian
```



(see github.com/shade-econ/sequence-jacobian)

Why is a toolkit useful?

- ❖ Many computations are **common across models**:
 - ❖ Het-agent models: lotteries for distribution, backward and forward iteration for steady state, fake news algorithm for Jacobians, ...
 - ❖ All models: differentiate equations, write as a linear system in sequence space, apply chain rule, solve the system, ...
- ❖ Coding these manually gets tedious as models become more complex, and **errors become more likely!** [Even now with AI...]
- ❖ Toolkit philosophy: automate everything **that's easy to automate**, leave some **high-level work to the user**
- ❖ See also related toolkits: HARK, BASEforHANK, econpizza, and more

SSJ toolkit essentials

Concept 1: blocks

- ❖ Toolkit is built around **blocks**
 - ❖ Each block is a piece of an economic model
 - ❖ It's a mapping from aggregate **inputs** to aggregate **outputs**
- ❖ Four main kinds of blocks:
 - ❖ **SimpleBlock**: simple aggregate equations, can have time leads or lags
 - ❖ **HetBlock**: heterogeneous agents that collectively take in some aggregate inputs (e.g. interest rates) and produce some outputs (e.g. consumption)
 - ❖ **SolvedBlock**: wraps around another block, solves for “unknowns” given “targets”
 - ❖ **CombinedBlock**: composition of many blocks

Example: SimpleBlock

- ❖ A SimpleBlock is defined like an ordinary Python function, decorated with @sj.simple
- ❖ Can put Dynare-style time offsets in parentheses: here, production is $Y_t = A_t K_{t-1}^\alpha L_t^{1-\alpha}$
- ❖ Block inputs are the function inputs; outputs must be listed in the return statement

```
import sequence_jacobian as sj
@sj.simple
def prod(A, K, L, alpha):
    Y = A * K(-1)**alpha * L**(1 - alpha)
    MPK = alpha * Y / K(-1)
    MPL = (1 - alpha) * Y / L
    return Y, MPK, MPL
```

```
print(prod), print(prod.outputs), print(prod.inputs)
```

✓ 0.0s

```
<SimpleBlock 'prod'>
['Y', 'MPK', 'MPL']
['A', 'K', 'L', 'alpha']
```

Concept 2: methods

- ❖ Each block has four standard **methods** that can be called on it:
 - ❖ **.steady_state()**
 - ❖ **.impulse_linear()**
 - ❖ **.impulse_nonlinear()**
 - ❖ **.jacobian()**
- ❖ Each also has a “solve” variant, e.g. **.solve_steady_state()** solves for steady state “unknowns” to hit certain steady-state “targets”

.steady_state() on our example

- ❖ .steady_state() is called on a dict giving the steady-state value of every input to a block, and returns a dict that adds every steady-state output
- ❖ For a SimpleBlock, it essentially evaluates the function, ignoring time offsets
- ❖ For other blocks, it's more complicated.
- ❖ Returns a SteadyStateDict, which is mostly a dict but with some enhancements (e.g. stores extra “internal” data for HetBlocks)

```
calibration = {'A': 1, 'K': 4, 'L': 1, 'alpha': 0.3}
ss = prod.steady_state(calibration)
ss
✓ 0.0s
<SteadyStateDict: ['A', 'K', 'L', 'alpha', 'Y', 'MPK', 'MPL']>
```

```
dict(**ss)
✓ 0.0s
{'A': 1,
 'K': 4,
 'L': 1,
 'alpha': 0.3,
 'Y': 1.515716566510398,
 'MPK': 0.11367874248827985,
 'MPL': 1.0610015965572785}
```


.solve_steady_state() on our example

- ❖ We might not know every steady-state parameter, but instead want to hit some calibration targets
- ❖ Then, we use .solve_steady_state(), specifying the “unknown” parameters and the “targets” we want to hit
- ❖ It’s a thin wrapper around .steady_state(), with an outer loop that does a nonlinear solver
 - ❖ can also do this manually, and sometimes should to have more control!

```
ss = prod.solve_steady_state(calibration,  
                             unknowns={'A': 1., 'alpha': 0.5},  
                             targets={'Y': 1, 'MPK': 0.1})
```

```
dict(**ss)
```

✓ 0.0s

```
{'A': np.float64(0.5743491774985149),  
 'K': 4,  
 'L': 1,  
 'alpha': np.float64(0.400000000000002894),  
 'Y': 1.00000000000000355,  
 'MPK': 0.100000000000001079,  
 'MPL': 0.59999999999999923}
```


.jacobian() on our example

- ❖ .jacobian() returns the sequence-space Jacobians of some outputs vs. inputs, as a **JacobianDict** (a nested dict with some enhanced features)
- ❖ [In most cases, this is numerical up to some horizon “T”, but for SimpleBlocks the default is to exploit the special structure of the Jacobian, returning as a “SimpleSparse” (which can act like a regular matrix)]

```
J = prod.jacobian(ss, inputs=['K', 'L'],  
                  outputs=['Y', 'MPL'])
```

✓ 0.0s

<JacobianDict outputs=['Y', 'MPL'], inputs=['K', 'L']>

```
J['Y', 'K']
```

✓ 0.0s

SimpleSparse({(-1, 0): 0.100})

```
J['Y', 'K'].matrix(5)
```

✓ 0.0s

```
array([[0. , 0. , 0. , 0. , 0. ],  
       [0.1, 0. , 0. , 0. , 0. ],  
       [0. , 0.1, 0. , 0. , 0. ],  
       [0. , 0. , 0.1, 0. , 0. ],  
       [0. , 0. , 0. , 0.1, 0. ]])
```


Applying JacobianDicts to impulses

- ❖ A JacobianDict can operate on impulses to its inputs (given as a dict), and calculate the resulting impulses to outputs
- ❖ The result is an ImpulseDict, again a kind of enhanced dict
- ❖ Can add and subtract ImpulseDicts, multiply them by scalars, multiply them by additional JacobianDicts, etc.

```
dL = 0.5 * 0.9**np.arange(25)
dK = 2 * 0.5**np.arange(25)
impulse = J @ {'K': dK, 'L': dL}
impulse
```

✓ 0.0s

```
<ImpulseDict: ['K', 'L', 'Y', 'MPL']>
```

```
impulse['K'][:5], impulse['Y'][:5]
```

✓ 0.0s

```
(array([2.    , 1.    , 0.5   , 0.25  , 0.125]),
 array([0.3    , 0.47   , 0.343  , 0.2687 , 0.22183]))
```


Or get in one shot with impulse methods

- ❖ `.impulse_linear()` is a more direct route to what we just did: rather than calculating the Jacobian and applying it to an impulse, it directly obtains the linearized output response
- ❖ `.impulse_nonlinear()` is the same, but does not linearize: it calculates the fully nonlinear impulse to outputs from the impulse to inputs

```
impulse2 = prod.impulse_linear(ss,  
| inputs={'K': dK, 'L': dL}, outputs=['Y', 'MPL'])  
impulse2['Y'][:5]
```

✓ 0.0s

```
array([0.3      , 0.47     , 0.343    , 0.2687   , 0.22183])
```

```
impulse3 = prod.impulse_nonlinear(ss,  
| inputs={'K': dK, 'L': dL}, outputs=['Y', 'MPL'])  
impulse3['Y'][:5]
```

✓ 0.0s

```
array([0.2754245 , 0.46979683, 0.3408179 , 0.26312152, 0.21467539])
```

[The `impulse_linear` response is exactly the same as on the previous slide. The `impulse_nonlinear` response is a bit smaller, because the shocks are huge and trigger nonlinearities.]

Concept 3: combining and solving blocks

- ❖ The toolkit becomes more powerful once we start **combining blocks**
- ❖ This composes them together, so that one block's output can be another's input
- ❖ The resulting “CombinedBlock” is treated as a single block that produces all outputs
- ❖ Here, we compose our “prod” block with a labor supply block and a block representing labor market equilibrium (setting the wage equal to MPL)

```
@sj.simple
def labor_supply(Lbar, w):
    L = Lbar * w**0.5
    return L

@sj.simple
def labor_mkt_clearing(w, MPL):
    labor_mkt = w - MPL
    return labor_mkt

model = sj.combine([prod, labor_supply, labor_mkt_clearing],
                    name='model')
model
✓ 0.0s
```

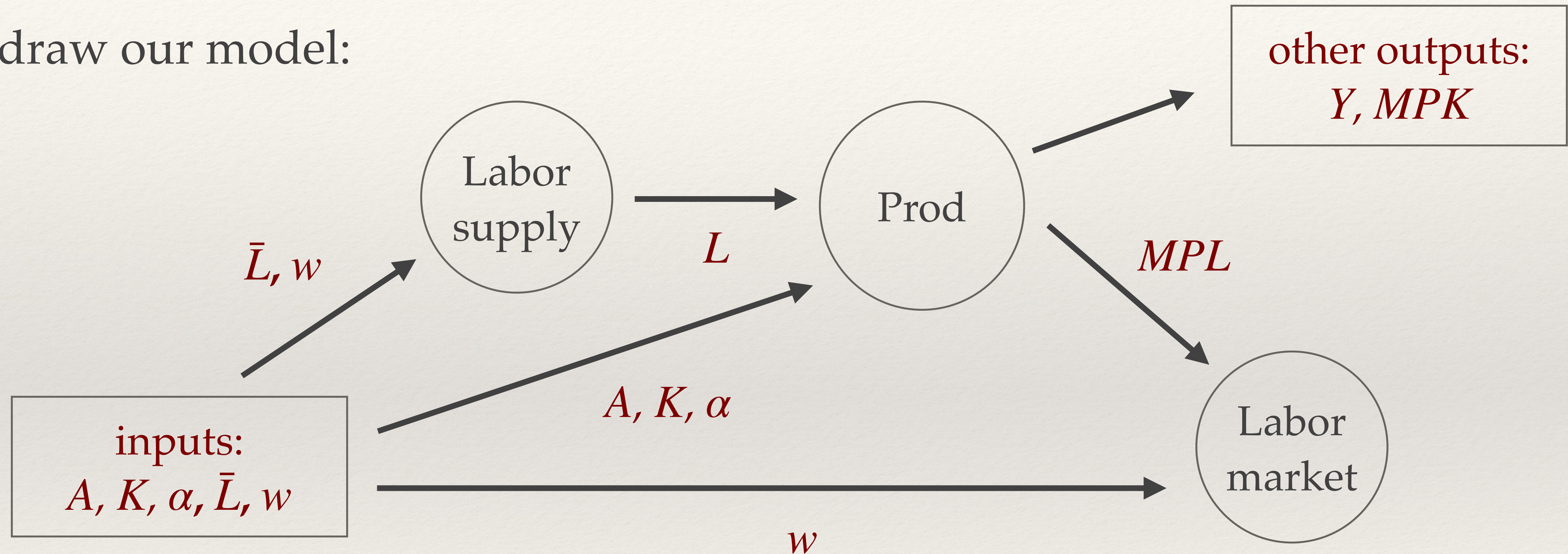
<CombinedBlock 'model'>

```
print(model.blocks)
print(model.inputs)
print(model.outputs)
✓ 0.0s
```

```
[<SimpleBlock 'labor_supply'>, <SimpleBlock 'prod'>, <SimpleBlock 'labor_mkt.
['A', 'K', 'alpha', 'Lbar', 'w']
['Y', 'MPK', 'MPL', 'L', 'labor_mkt']
```


Directed Acyclic Graphs (DAGs)

- ❖ Can draw our model:



- ❖ The toolkit does this **automatically**, placing and evaluating blocks in the right order

Solving the model

- ❖ We can play with this model as a whole
- ❖ For instance, we can calibrate many parameters to hit targets using `.solve_steady_state()`
- ❖ Then, with the resulting steady state, we can use `.solve_jacobian()`, finding how various outcomes (Y, L, w) respond to exogenous parameters ($A, \bar{L}$), when we solve for the market-clearing w
- ❖ Under the hood, the toolkit is building a linear system using the chain rule, and solving it!

```
ss = model.solve_steady_state(calibration,  
    |         unknowns={'A': 1, 'alpha': 0.5, 'Lbar': 1, 'w': 1},  
    |         targets={'Y': 1, 'L': 1, 'MPK': 0.1, 'labor_mkt': 0})  
ss  
✓ 0.0s
```

```
<SteadyStateDict: ['A', 'K', 'L', 'alpha', 'Lbar', 'w', 'Y', 'MPK',
```

```
J = model.solve_jacobian(ss,  
    |         inputs=['A', 'Lbar'], outputs=['Y', 'L', 'w'],  
    |         unknowns=['w'], targets=['labor_mkt'], T=400)  
J  
✓ 0.0s
```

```
<JacobianDict outputs=['w', 'Y', 'L'], inputs=['A', 'Lbar']>
```

```
J['Y', 'A'][:3, :3].round(3)
```

```
✓ 0.0s  
  
array([[2.176, 0.    , 0.    ],  
       [0.    , 2.176, 0.    ],  
       [0.    , 0.    , 2.176]])
```


This gets harder without a toolkit...

- ❖ The overall Jacobian of Y to A , imposing market clearing, that we saw includes:
 - ❖ The direct sensitivity of Y to A
 - ❖ The sensitivity of Y to L , combined with the sensitivity of L to w , combined with the sensitivity of w to A under market clearing
- ❖ On the right we do this manually, but it takes more effort (and we're cheating by taking the sensitivity of w to A we already calculated)!
- ❖ As models become more complicated, probability of a mistake with manual calculation $\longrightarrow 1$

```
Y_AL = prod.jacobian(ss, inputs=['L', 'A'], outputs=['Y'])
Y_A, Y_L = Y_AL['Y', 'A'], Y_AL['Y', 'L']
L_w = labor_supply.jacobian(ss, inputs=['w'], outputs=['L'])['L', 'w']
L_A = L_w @ J['w', 'A']
Y_AL = Y_L @ L_A + Y_A
Y_AL[:3, :3].round(3)
```

✓ 0.0s

```
array([[2.176, 0.    , 0.    ],
       [0.    , 2.176, 0.    ],
       [0.    , 0.    , 2.176]])
```


Concept 4: HetBlocks

- ❖ **HetBlocks** represent heterogeneous agents
- ❖ Define with backward iteration function
- ❖ Takes in the forward-looking “backward” variable (here, `Va_p`) plus other inputs
- ❖ Returns this period’s backward variable (here, `Va`), plus the policy variable (here, assets ‘`a`’) and other variables of interest
- ❖ Supply key information about hetblock, including the matrix or matrices (here, `Pi`) giving exogenous state transitions, and an initializer function (here, `hh_init`)

```
def hh_init(a_grid, y, r, eis):  
    coh = (1 + r) * a_grid[np.newaxis, :] + y[:, np.newaxis]  
    Va = (1 + r) * (0.1 * coh) ** (-1 / eis)  
    return Va  
  
@het(exogenous='Pi', policy='a', backward='Va', backward_init=hh_init)  
def hh(Va_p, a_grid, y, r, beta, eis):  
    uc_nextgrid = beta * Va_p  
    c_nextgrid = uc_nextgrid ** (-eis)  
    coh = (1 + r) * a_grid[np.newaxis, :] + y[:, np.newaxis]  
    a = interpolate.interpolat_y(c_nextgrid + a_grid, coh, a_grid)  
    misc.setmin(a, a_grid[0])  
    c = coh - a  
    Va = (1 + r) * c ** (-1 / eis)  
    return Va, a, c
```

[This is code for the built-in `sj.hetblocks.hh_sim.hh` model in the toolkit, which we’ll use most often. It’s very similar to our fast SIM code, just with slightly different function names, and it doesn’t apply `Pi` since the toolkit takes care of that.]

HetBlocks continued

- ❖ All methods can be applied to HetBlocks
- ❖ So under the hood, the toolkit can do a **ton**: solve for steady state, iterate backward and forward nonlinearly, apply fake news algorithm to get Jacobian, etc...
- ❖ For now, we allow arbitrarily many discrete exogenous states (transitions given by ‘exogenous’), then one or two endogenous continuous states (by ‘policy’)
 - ❖ Exogenous states ordered first, then endogenous states, in dimensions of return variables
- ❖ Extension under development (“StageBlock”) that’s far more general...

```
def hh_init(a_grid, y, r, eis):  
    coh = (1 + r) * a_grid[np.newaxis, :] + y[:, np.newaxis]  
    Va = (1 + r) * (0.1 * coh) ** (-1 / eis)  
    return Va  
  
@het(exogenous='Pi', policy='a', backward='Va', backward_init=hh_init)  
def hh(Va_p, a_grid, y, r, beta, eis):  
    uc_nextgrid = beta * Va_p  
    c_nextgrid = uc_nextgrid ** (-eis)  
    coh = (1 + r) * a_grid[np.newaxis, :] + y[:, np.newaxis]  
    a = interpolate.interpolat_y(c_nextgrid + a_grid, coh, a_grid)  
    misc.setmin(a, a_grid[0])  
    c = coh - a  
    Va = (1 + r) * c ** (-1 / eis)  
    return Va, a, c
```

```
print(hh)  
print(hh.inputs)  
print(hh.outputs)
```

✓ 0.0s

```
<HetBlock 'hh'>  
['a_grid', 'y', 'r', 'beta', 'eis', 'Pi']  
['A', 'C']
```


Brief addendum: “hetinputs”

- ❖ The default “hh” HetBlock on the last slide takes a vector “y” of incomes
- ❖ In our basic GE model, these incomes are the product of after-tax aggregate income Z_t and labor endowment e
- ❖ Could write a new HetBlock with Z and e as inputs rather than y ...
- ❖ ... but more convenient to take the existing HetBlock, and add a new function that remaps y to other inputs
- ❖ Called a “hetinput” (also have “hetoutput”)

```
def income(e, Z):  
    y = Z * e  
    return y
```

```
hh = hh.add_hetinputs([income])  
print(hh.inputs)
```

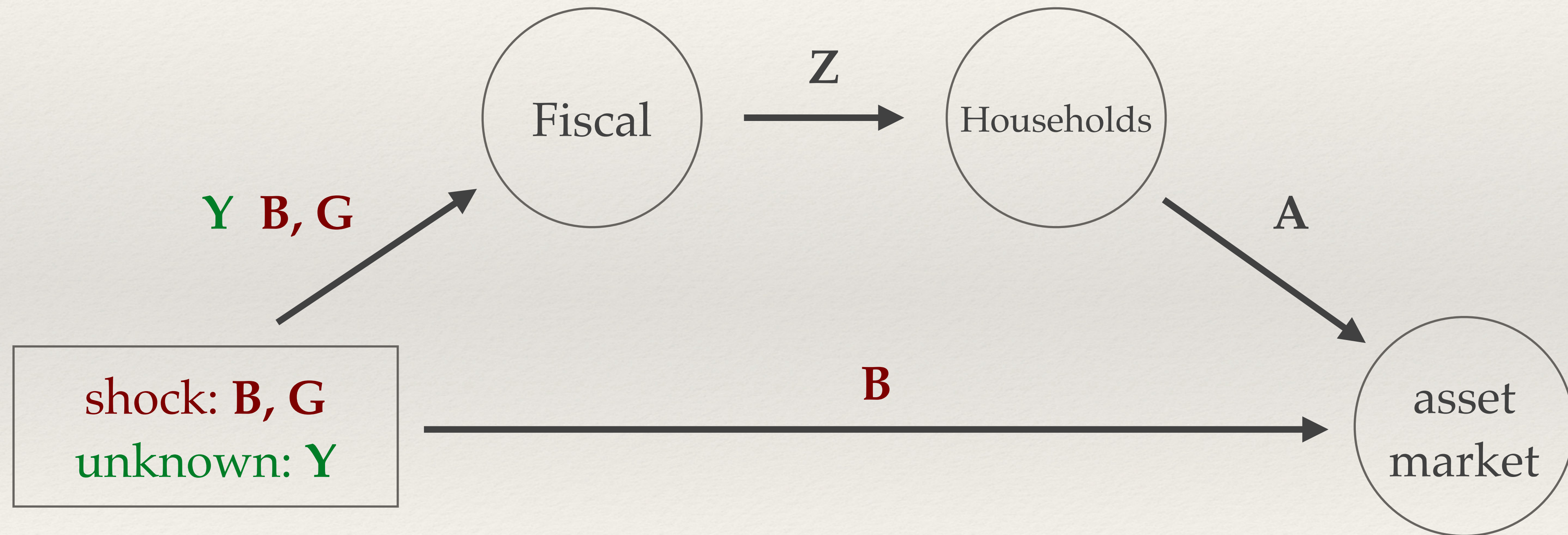
✓ 0.0s

```
['a_grid', 'r', 'beta', 'eis', 'Pi', 'e', 'Z']
```


Application 1: redo fiscal policy exercise

Solving for output response of shocks

- ❖ Holding r and other parameters constant, want Y response to B, G shocks



- ❖ Last lecture we derived this analytically, now let's write generic SSJ

Writing in SSJ toolkit

```
@sj.simple
def fiscal(B, r, G, Y):
    T = (1 + r) * B(-1) + G - B
    Z = Y - T
    return T, Z

@sj.simple
def mkt_clearing(A, B, Y, C, G):
    asset_mkt = A - B
    goods_mkt = Y - C - G
    return asset_mkt, goods_mkt

model = sj.combine([hh, fiscal, mkt_clearing])
```

```
from calibration import make_calibration
calib, e = make_calibration(lowA=True)
calib.update({'e': e, 'B': 4,
              'T': 0.3, 'Y': 1,
              'G': 0.3 - calib['r']*4})

ss = model.steady_state(calib)
assert np.isclose(ss['asset_mkt'], 0, atol=1E-7)
assert np.isclose(ss['goods_mkt'], 0, atol=1E-7)
```

Since we already calculated a calibration outside the toolkit, we'll use that here, then verify that markets clear when we embed that into GE (note: bonds here are 100% of annual GDP, 400% of quarterly.)

Alternatively, could have used toolkit for everything.

Now solution is a single line of code!

- ❖ In our model, we're solving for an unknown output sequence $\mathbf{Y}$ to hit asset market clearing, given a shock to the path of bonds $\mathbf{B}$
- ❖ which induces a change in transfers $\mathbf{T}$
- ❖ single line of SSJ, using `solve_impulse_linear`!
- ❖ can get the nonlinear solution for same shock, just `solve_impulse_nonlinear`!
- ❖ Finally, verify that it's the same as our direct linear solution with the "A" matrix.

```
imp_linear = model.solve_impulse_linear(ss,  
    unknowns=['Y'], targets=['asset_mkt'],  
    inputs={'B': dB})  
imp_linear['Y'][:5].round(3)  
✓ 1.7s
```

```
array([0.043, 0.042, 0.041, 0.04 , 0.038])
```

```
imp_nonlin = model.solve_impulse_nonlinear(ss,  
    unknowns=['Y'], targets=['asset_mkt'],  
    inputs={'B': dB}, verbose=False)  
imp_nonlin['Y'][:5].round(3)  
✓ 3.7s
```

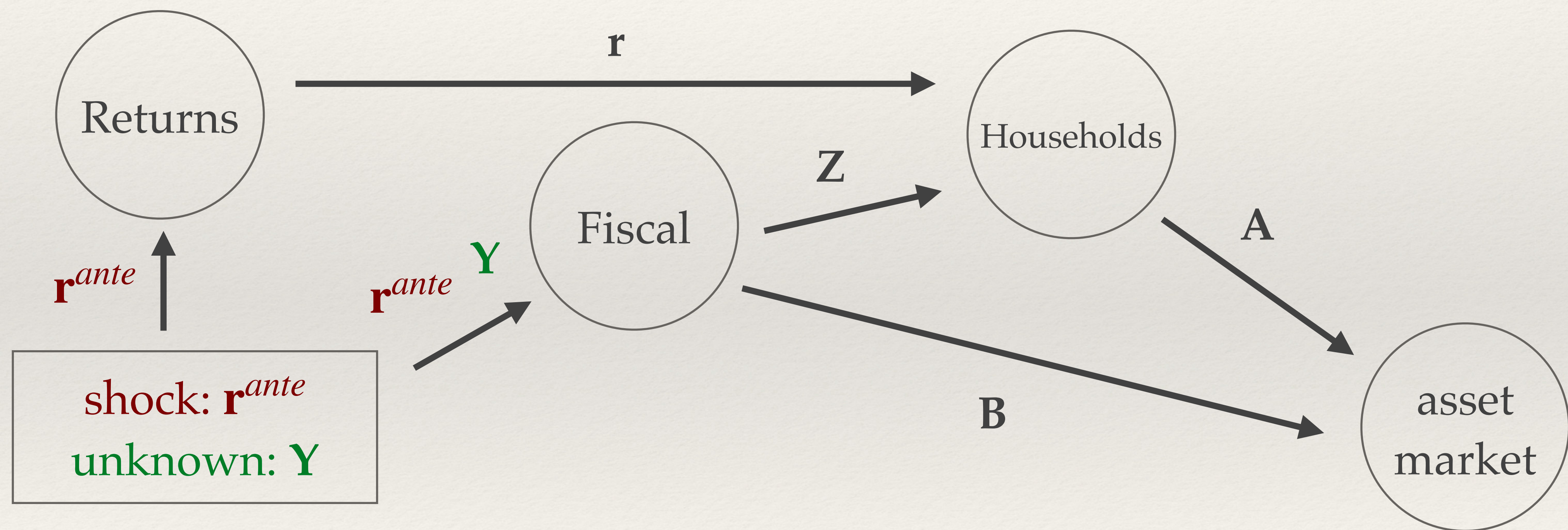
```
array([0.041, 0.04 , 0.039, 0.038, 0.037])
```

```
A = hh.jacobian(ss, inputs=['Z'],  
    outputs=['A'], T=400)['A', 'Z']  
dY = np.linalg.solve(A, dB) + dT  
assert np.allclose(dY, imp_linear['Y'])
```


Application 2: redo monetary policy exercise

Solving for output response of shocks

- ❖ Now we want to shock ex-ante real interest rates as in the last lecture



- ❖ Two complications: we need to shift r^{ante} to our ex-post r that household uses, and B is also (slightly) endogenous here, based on the rule that holds $(1 + r_t)B_t$ fixed

Writing in SSJ toolkit

```
@sj.simple
def returns(r_ante):
    r = r_ante(-1)
    return r

@sj.simple
def fiscal(r_ante, G, Y):
    B = ss['B'] * (1 + ss['r']) / (1 + r_ante)
    T = (1 + r_ante(-1)) * B(-1) + G - B
    Z = Y - T
    return T, Z, B

model = sj.combine([returns, fiscal, mkt_clearing, hh])
ss = model.steady_state(**ss, 'r_ante': ss['r'])
```

The market-clearing block is unchanged from before, and we can reuse essentially the same steady state as in fiscal case.

```
# scale shock as a 1pp (annualized) cut
# note vector "dr" from lecture 4 is dlog(1+r_ante)
dr = -0.0025 * 0.9**np.arange(T) / (1 + ss['r'])
imp_linear = model.solve_impulse_linear(ss,
    unknowns=['Y'], targets=['asset_mkt'],
    inputs={'r_ante': (1+ss['r'])*dr})
(imp_linear['Y'][:5]).round(3)
```

✓ 2.7s

```
array([0.022, 0.02 , 0.018, 0.017, 0.015])
```

```
imp_nonlin = model.solve_impulse_nonlinear(ss,
    unknowns=['Y'], targets=['asset_mkt'],
    inputs={'r_ante': (1+ss['r'])*dr}, verbose=False)
(imp_nonlin['Y'][:5]).round(3)
```

✓ 3.2s

```
array([0.022, 0.02 , 0.018, 0.017, 0.015])
```

No visible nonlinearity for this shock!

Final application:
add Taylor rule to fiscal policy case

Write blocks for NKPC, Taylor rule

```
@sj.simple
def fiscal(B, r, G, Y):
    # note: taking B as exogenous again
    T = (1 + r) * B(-1) + G - B
    Z = Y - T
    return T, Z

beta_mean = ss['beta'].mean()
@sj.solved(unknowns={'pi': (-0.5, 0.5)},
           targets=['nkpc_resid'])
def pricing(pi, kappa, Y):
    nkpc_resid = (pi - kappa*(Y - ss['Y'])
                  - beta_mean*pi(+1))
    return nkpc_resid

@sj.simple
def taylor(pi, phi_pi, r_ante):
    i = ss['r'] + phi_pi * pi
    r_resid = i - pi(+1) - r_ante
    return r_resid
```

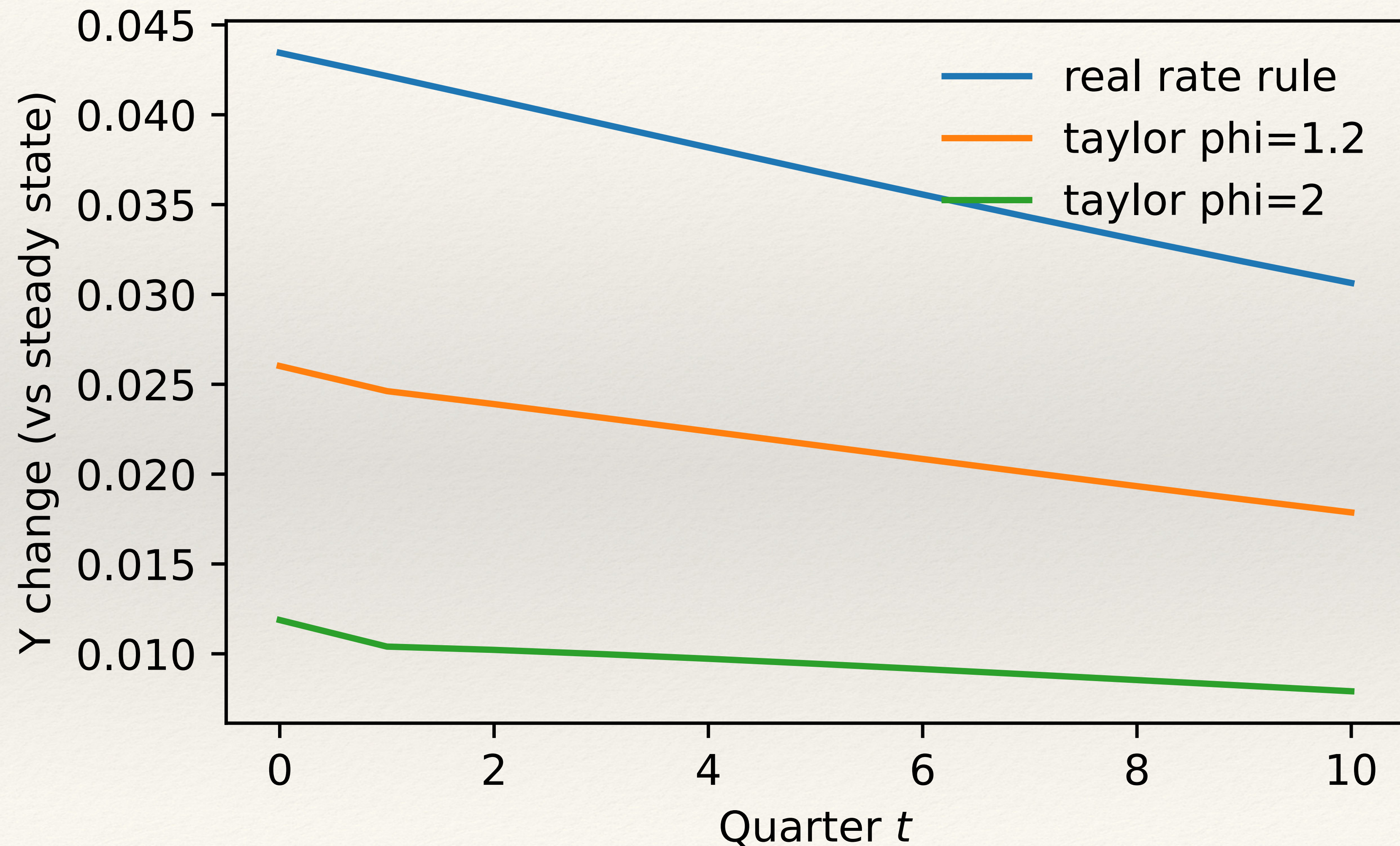
```
model = sj.combine([hh, returns, fiscal,
                    pricing, taylor, mkt_clearing])

ss = model.steady_state(**ss,
                        'kappa': 0.01, 'phi_pi': 1.2)

dY_phi_12 = model.solve_impulse_linear(ss,
                                       unknowns=['Y', 'r_ante'],
                                       targets=['asset_mkt', 'r_resid'],
                                       inputs={'B': dB})['Y']
```

Now need to define a “pricing” block, which solves for pi to hit the NKPC, and a Taylor rule. In the main loop, solve for another unknown, r_ante, to be consistent with the Taylor rule and Fisher equation.

Effect of tax cut shock by monetary policy



Conclusion

Concluding thoughts

- ❖ We can always work manually with Jacobians, but the math quickly gets complicated!
 - ❖ Lots of variables to handle, manually doing the fake news algorithm, manually doing the chain rule...
- ❖ The SSJ toolkit offers a systematic way to avoid this drudgery
- ❖ It's also a useful **language** in which to think about writing models
- ❖ Still some bugs and quirks that we'll work to fix, but with AI it's easier than ever to use